МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1
по курсу «ПГП и ТПП»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами

Выполнил: Кудрявов Л.В.
Группа: М8О-406Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

1. **Цель работы.** Ознакомление с программным обеспечением для работы с программно-аппаратной архитектурой параллельных вычислений CUDA и его установка. Реализация одной из примитивных операций над векторами. Вещественный тип данных — `double`, результат выводится с относительной точностью 10^{-10}, размер векторов $n < 2^{25}$.
2. **Вариант 2. Вычитание векторов.** Входные данные: на первой строке число n — размер векторов, в следующих двух строках по n вещественных чисел — элементы векторов. Выходные данные: n чисел — результат вычитания исходных векторов. Пример: для входа «3 / 1 2 3 / 4 5 6» выходом является строка `-3.0000000000e+00 -3.0000000000e+00 -3.0000000000e+00`.

Программное и аппаратное обеспечение

Вычисления выполнялись в облачной среде Google Colab с графическим ускорителем NVIDIA. Характеристики графического процессора приведены в таблице 1. Значения даны для NVIDIA Tesla T4 (compute capability 7.5, та же архитектура, что у GeForce GTX 1650 проверяющей системы). Характеристики соответствуют стандартной конфигурации Google Colab с ускорителем T4.

Таблица 1 — Характеристики оборудования

Параметр	Значение
Графический процессор	NVIDIA Tesla T4
Compute capability	7.5
Графическая (глобальная) память	15 360 МиБ (≈ 15 ГБ), GDDR6
Разделяемая память на блок	48 КБ (64 КБ на мультипроцессор)
Константная память	64 КБ
Регистров на блок	65 536
Максимум нитей в блоке	1024 (размеры блока до 1024 × 1024 × 64)
Максимальный размер сетки	$2^{31} - 1 \times 65\,535 \times 65\,535$ блоков
Число мультипроцессоров	40
Размер варпа	32 нити
Центральный процессор	Intel(R) Xeon(R) CPU @ 2.00GHz, 2 логических ядра
Оперативная память	≈ 12 ГБ
Дисковая память	≈ 100 ГБ

Проверяющая система курса: GeForce GTX 1650 (compute capability 7.5, 4 ГБ глобальной памяти, 48 КБ разделяемой памяти на блок, 65 536 регистров на блок), nvcc 12.0, g++ 13.3.0, Ubuntu 24.04.1 LTS.

Программное обеспечение: операционная система Ubuntu 24.04 (среда выполнения Google Colab), компилятор nvcc из состава CUDA Toolkit 13.0 (V13.0.88), компилятор g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), Python 3 с библиотеками NumPy, pandas, matplotlib и Pillow (генерация тестов, конвертер изображений,

независимая проверка результатов, таблицы). Среда разработки: ноутбуки Jupyter в Google Colab, текстовый редактор. Сдаваемые программы собираются командой `nvcc -arch=sm_75 -Werror cross-execution-space-call -lm`, профилирующие — `nvcc -O2 -arch=sm_75`. Версия nvcc проверяющей системы (12.0) отличается от использованной, но программы применяют только базовые возможности CUDA.

Метод решения

Требуется вычислить вектор res с элементами $res_i = a_i - b_i$, $i = 0, \ldots, n - 1$. Элементы результата не зависят друг от друга, поэтому задача полностью параллельна по данным (операция map): каждую пару элементов может обработать отдельная нить, а межнитевой синхронизации не требуется.

Вычислительная сложность алгоритма O(n). На один элемент приходится одна арифметическая операция и 24 байта обмена с глобальной памятью (два чтения и одна запись типа double), то есть арифметическая интенсивность очень мала. Поэтому время работы ограничено не вычислительной мощностью, а пропускной способностью памяти видеокарты и скоростью копирования данных по шине PCIe.

Схема работы программы: чтение n и векторов a, b с проверкой корректности ввода; выделение трёх массивов в памяти видеокарты (`cudaMalloc`); копирование a и b на устройство; запуск ядра; копирование результата на хост; вывод в формате `%.10e`; освобождение памяти.

Число нитей фиксировано и не зависит от n (требование курса): ядро запускается как `<<<256, 256>>>`, а нить обрабатывает элементы с шагом, равным общему числу нитей в сетке (grid-stride loop). Нити с соседними номерами обращаются к соседним элементам массивов, поэтому запросы к глобальной памяти объединяются (coalescing).

Использованные источники: Боресков А.В., Харламов А.А. Основы работы с технологией CUDA (М.: ДМК Пресс, 2010); Сандерс Дж., Кэндрот Э. Технология CUDA в примерах (М.: ДМК Пресс, 2011); NVIDIA CUDA C++ Programming Guide; лекции курса.

Описание программы

Программа состоит из одного файла `lab1.cu`. Основные элементы:

- `CSC(call)` — макрос проверки результата любого вызова CUDA API. При ошибке в stderr выводится сообщение `ERROR: …`, программа завершается с кодом 0 (требование курса).
- `kernel(a, b, res, n)` — ядро вычитания векторов (листинг ниже).
- `main` — чтение и проверка входных данных (ограничение $0 \le n < 2^{25}$, ошибки чтения), выделение памяти, копирование, запуск ядра, проверка `cudaGetLastError`, копирование результата, вывод.

```
__global__ void kernel(const double *a, const double *b, double *res, long
long n) {
```

```
    long long idx = (long long)blockIdx.x * blockDim.x + threadIdx.x;
    long long offset = (long long)blockDim.x * gridDim.x;
    while (idx < n) {
        res[idx] = a[idx] - b[idx];
        idx += offset;
    }
}
```

Индексы имеют тип `long long`, чтобы исключить переполнение при больших n. Результат записывается поверх первого вектора на хосте, отдельный буфер для вывода не нужен.

Результаты

1. Проверка корректности

Таблица 2 — Тесты программы

Тест	Ожидаемый результат	Результат
Пример из условия: n = 3, (1 2 3) − (4 5 6)	-3.0000000000e+00 -3.0000000000e+00 -3.0000000000e+00	совпадает
Большие и малые значения: (1e300, −2,5, 0,1) − (−1e300, 5, 0,3)	2.0000000000e+300 -7.5000000000e+00 -2.0000000000e-01	совпадает
n = 0	пустая строка	совпадает
Нет входных данных	ERROR в stderr, код возврата 0	совпадает
Недостаточно чисел во втором векторе	ERROR в stderr, код возврата 0	совпадает
n = 2^{25} (за границей ограничения)	ERROR в stderr, код возврата 0	совпадает

Корректность ядра на GPU подтверждена и при замерах: результат сравнивался с CPU-реализацией, максимальное расхождение равно 0 для всех конфигураций и размеров.

2. Время работы ядра при разных конфигурациях запуска

Замеры выполнены профилирующей программой `lab1_bench.cu`: ядро то же, что в `lab1.cu`, входные векторы из n случайных чисел генерируются внутри программы, сборка `nvcc -O2 -arch=sm_75`. Время ядра измерялось событиями `cudaEvent` и усреднялось по 5 запускам после прогрева, конфигурация `<<<blocks, threads>>>` задавалась параметрами. Сдаваемая версия использует `<<<256, 256>>>`. Результаты приведены в таблице 3.

Таблица 3 — Время работы ядра, мс

Конфигурация	n = 10^3	n = 10^5	n = 10^7	n = $2^{25} - 1$
<<<1, 32>>>	0,0201	0,873	115,40	390,20
<<<1, 256>>>	0,0040	0,0814	15,04	50,43
<<<1, 1024>>>	0,0041	0,0409	4,659	15,54
<<<16, 256>>>	0,0038	0,0076	1,348	4,537
<<<64, 256>>>	0,0041	0,0050	0,963	3,271
<<<256, 256>>>	0,0043	0,0052	0,979	3,290

Конфигурация	$n = 10^3$	$n = 10^5$	$n = 10^7$	$n = 2^{25} - 1$
<<<1024, 256>>>	0,0048	0,0066	0,977	3,265
<<<1024, 1024>>>	0,0127	0,0179	0,965	3,233

При $n = 2^{25} - 1$ время ядра уменьшается с 390,20 мс для `<<<1, 32>>>` до 3,271 мс для `<<<64, 256>>>`, то есть в 119 раз. Начиная примерно с 64 блоков по 256 нитей время перестаёт уменьшаться (`<<<1024, 1024>>>`: 3,233 мс): ядро упирается в пропускную способность памяти. Конфигурации с одним блоком заметно медленнее (`<<<1, 1024>>>`: 15,54 мс): блок целиком выполняется на одном мультипроцессоре из 40, и один мультипроцессор не может загрузить память видеокарты; `<<<16, 256>>>` задействует 16 мультипроцессоров (4,537 мс). При $n = 10^3$ время всех конфигураций составляет единицы микросекунд — это накладные расходы запуска ядра, а избыточные сетки (`<<<1024, 1024>>>`: 0,0127 мс) даже немного хуже.

Эффективная пропускная способность: ядро читает два и записывает один массив типа double ($3 \cdot 8 \cdot n \approx 805$ МБ) за 3,272 мс, то есть около 246 ГБ/с. Это около 77% от пиковой пропускной способности памяти Tesla T4 (≈ 320 ГБ/с).

3. Сравнение с CPU

Для сравнения в той же программе выполняется последовательная версия на CPU: цикл `res[i] = a[i] - b[i]` в одном потоке (процессор Intel Xeon 2,00 ГГц, 2 логических ядра, использовалось одно). Время CPU — среднее по 5 проходам для $n \le 10^6$, по 3 проходам для $n \le 10^7$, для большего n — один проход. Ускорение — отношение времени CPU к полному времени GPU, включающему копирование векторов на видеокарту и результата обратно (таблица 4, рисунок 1).

Таблица 4 — Сравнение CPU и GPU

Размер данных	CPU, мс	GPU, ядро, мс	GPU, с копированием, мс	Ускорение, раз
$n = 10^5$	0,193	0,0077	1,175	0,164
$n = 10^6$	2,087	0,0986	9,522	0,219
$n = 10^7$	29,83	0,972	91,44	0,326
$n = 2^{25} - 1$	156,89	3,272	306,61	0,512

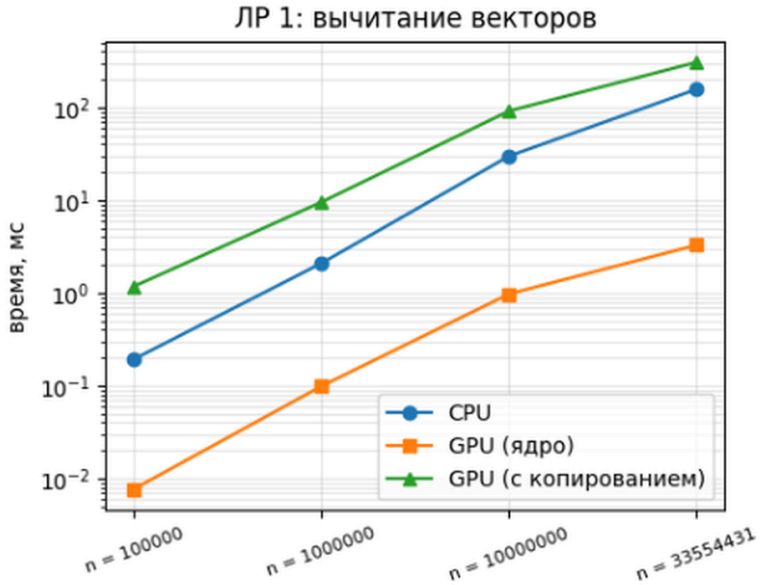


Рисунок 1 — Время вычитания векторов на CPU и GPU

Ядро на GPU быстрее CPU в 25, 21, 31 и 48 раз (с ростом n), но полное время GPU с копированием в 2–6 раз больше времени CPU (ускорение 0,164–0,512 в зависимости от n). Копирование 805 МБ (два вектора на устройство и результат обратно) заняло около 303 мс, то есть 2,7 ГБ/с через PCIe из обычной (pageable) памяти хоста, что в 93 раз дольше самого вычисления. Для ускорения копирования можно использовать закреплённую память (`cudaMallocHost`) и перекрытие копирования с вычислениями при помощи потоков; в работе это не применялось.

Выводы

Вычитание векторов — базовая операция линейной алгебры (наряду со сложением и операциями вида y = a·x + y в BLAS первого уровня), которая встречается в обработке сигналов, численных методах и машинном обучении. Она служит типичным примером поэлементной (map) задачи, хорошо подходящей для GPU.

Сложность программирования невысока: основные трудности связаны с настройкой окружения, обязательной проверкой ошибок CUDA, соблюдением требования о фиксированном числе нитей (решается циклом с шагом) и точностью вывода (формат `%.10e`).

Эксперименты на Tesla T4 показали, что время ядра определяется конфигурацией запуска: при $n = 2^{25} - 1$ переход от `<<<1, 32>>>` к `<<<64, 256>>>` ускоряет ядро в 119 раз, дальнейшее увеличение сетки эффекта не даёт, так как достигнута пропускная способность памяти (около 246 ГБ/с). Само ядро быстрее последовательной версии на CPU в 21–48 раз, однако с учётом копирования по PCIe GPU в этой задаче проигрывает CPU (ускорение 0,164–0,512): арифметическая интенсивность операции мала (одно вычитание на 24 байта данных), и время определяется передачей данных. Применять GPU для такой операции оправдано, когда данные уже находятся в памяти видеокарты или обрабатываются цепочкой операций.